AGENT EVALUATION SCIENCE

Evaluating Agent Swarms: Token Budgets, Active Workers, and Baseline Choice

Maximilian Nicholson[1,2] Ayda Yazdani[1]
[1] University of Bath [2] Amazon
mn866@bath.ac.uk ay582@bath.ac.uk

Abstract

Evaluating an agent swarm requires distinguishing the effects of increasing the total token budget, dividing a fixed ceiling, and changing coordination. We examine these comparisons in 8,400 controlled trials on fresh arithmetic and graph-colouring tasks. At 4,096 completion tokens per worker, increasing independent width from one to four improves accuracy by 16.0 percentage points on both families while the total ceiling quadruples. Splitting a fixed 4,096-token completion ceiling instead lowers graph accuracy from 56.0% to 18.5%; the arithmetic change is inconclusive. The primary comparisons do not establish an accuracy advantage over a sequential control matched on scheduled calls and nominal token ceiling. Ring communication provides no consistent gain. Eleven complementary scaling studies add 20,376 held-out trials across seven task families, including a fresh-task sixteen-worker confirmation that does not establish superiority over fixed-four execution. Candidate and request audits show that finding a valid answer after an initial failure depends on the token allowance. They also reveal incomplete worker activation and 97 controlled-study attempts with unknown usage. The findings show how resource controls, output selection, and execution records qualify claims about agent scaling. They support evaluating specific allocation policies; equal realized compute advantages and general scaling laws remain unestablished from this work.

Keywords: Agent evaluation; Multi-agent systems; Test-time compute; Token budgets; Resource allocation; Coordination; Reliability; Reproducibility

> "All happy families are alike; each unhappy family is unhappy in its own way." —Leo Tolstoy, *Anna Karenina* (Tolstoy, 2014).
>
> *Adapted to this study:* All successful swarms share a valid outcome; unsuccessful swarms fail in different ways.

1 Evaluation Design

Worker-count evaluations combine resource allocation, candidate selection, and communication. Prior work on sampling and inference allocation motivates separating these factors (Brown et al., 2024; Snell et al., 2024).

The controlled study contains 8,400 trials on fresh arithmetic and graph-colouring tasks at widths $N = 1, 2, 4$. With per-worker completion allowance d, total ceiling $B = Nd$, and two calls per worker, each call receives $d/2$ tokens. Independent workers revise their own drafts; ring workers receive predecessor information. A sequential control uses one worker for the same $2N$ scheduled calls and nominal ceiling, with prior-candidate and verifier feedback. All scheduled calls execute; the first valid candidate is retained.

Eleven complementary studies contribute 20,376 evaluation and 6,272 calibration trials across seven task families. They use programming benchmarks (Zhuo et al., 2025; Austin et al., 2021; Liu et al., 2023), ZebraLogic (Lin et al., 2025), BBEH (Kazemi et al., 2025), PlanBench (Valmeekam et al., 2023), and custom graph and Countdown tasks. The programming and

original ZebraLogic grids test widths 1, 2, 4, 8, and 16; the smaller diagnostic grids compare one and four. Generated graph and arithmetic tasks use deterministic constraint checks. Selection varies between verifier-assisted search, model integration, and gold-blind plurality. Paired bootstrap analyses retain repeated observations within task clusters; intervals are study-specific without cross-study adjustment.

2 Findings

Table 1 separates increasing the total allowance from dividing a fixed ceiling. The fixed-total arithmetic comparison remains inconclusive despite its negative point estimate. The primary comparisons establish neither an accuracy advantage over matched sequential execution nor a consistent ring-communication gain.

Independent width 1 to 4	Arithmetic	Graph colouring
Fixed per-worker $d = 4{,}096$	$+16.0$ pp	$+16.0$ pp
Fixed total $B = 4{,}096$	-24.0 pp	-37.5 pp

Table 1 | Accuracy changes in percentage points. Holding d fixed quadruples B. The fixed-total arithmetic contrast remains inconclusive.

All 148 wider complementary-study cells use more recorded tokens than their matched-ceiling single-worker controls: accuracy is lower in 75, equal in 24, and higher in 49. These are descriptive counts from unequal grids sharing tasks and controls.

PlanBench illustrates how the width effect can reverse with the allowance. At an 8,192-token ceiling, four independent workers score 0% versus 95% for one; at 16,384, they score 100% versus 89.5%, using 4.06 times the single-worker retained debit. Width and per-worker allowance change together, so the reversal does not isolate a causal mechanism.

A fresh-task ZebraLogic confirmation transfers a ring $4/8/16$ policy to 300 unused puzzles; against fixed four at the two higher ceilings, its accuracy difference is $+0.92$ pp (95% interval $[-0.58, 2.50]$). This establishes neither superiority nor equivalence.

3 Interpretation and Reproducibility

Equal nominal ceilings do not establish equal realized compute. Retained charges are incomplete attempt ledgers; the controlled study includes 97 attempts with unknown usage. Recovery after initial failure depends on the allowance. Heterogeneous models, small programming task sets, operational amendments, and possible public-task contamination limit generalization. The findings support evaluating specific allocation and selection policies; general scaling laws remain unestablished.

Extended methods, all 176 complementary evaluation cells, saved outcomes, recorded configurations, provenance, and an offline verifier are available in the repository.[1] The controlled-study summary includes an explicit evidence-coverage note. The verifier covers the eleven complementary studies, without recomputing intervals, validating graders, or replaying inference.

Funding and interests. This work was financially supported in part by OpenAI and Daytona. The sponsors neither exercised editorial control nor requested specific findings. The content and conclusions remain solely the responsibility of the authors.

[1] https://github.com/PluvioXO/agent-swarms-reproducibility

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, et al. Program synthesis with large language models, 2021. https://arxiv.org/abs/2108.07732. Cited at: 1.

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, et al. Large language monkeys: Scaling inference compute with repeated sampling, 2024. https://arxiv.org/abs/2407.21787. Cited at: 1.

Mehran Kazemi, Bahare Fatemi, Hritik Bansal, John Palowitch, Chrysovalantis Anastasiou, Sanket Vaibhav Mehta, et al. Big-bench extra hard, 2025. https://arxiv.org/abs/2502.19187. Cited at: 1.

Bill Yuchen Lin, Ronan Le Bras, Kyle Richardson, Ashish Sabharwal, Radha Poovendran, Peter Clark, et al. Zebralogic: On the scaling limits of llms for logical reasoning, 2025. https://arxiv.org/abs/2502.01100. Cited at: 1.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation, 2023. https://arxiv.org/abs/2305.01210. Cited at: 1.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024. https://arxiv.org/abs/2408.03314. Cited at: 1.

Leo Tolstoy. *Anna Karenina*. Penguin Classics, 2014. ISBN 9780141199610. https://www.penguinrandomhouse.com/books/530530/anna-karenina-by-leo-tolstoy/9780141199610/. Translated by Richard Pevear and Larissa Volokhonsky; Part One, Chapter 1. Cited at: 1.

Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change, 2023. https://arxiv.org/abs/2206.10498. Cited at: 1.

Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions, 2025. https://arxiv.org/abs/2406.15877. Cited at: 1.